I. CSWeb8 transformation process

A. Integrate pdo_pgsql driver

- ✓ In DictionarySchemaHelper.php file
- ✓ In DataSettings.php file
- ✓ In MapDataRepository.php file

B. DataBase updating

- ✓ Execute « ALTER TABLE `cspro_dictionaries_schema` DROP INDEX `schema_name`; » in CSWeb database

C. Scripts updating

- ✓ Transform « cleanDictionarySchema » function :

From :

```
    private function cleanDictionarySchema() {
        try {
            $tables = $this->conn->getSchemaManager()->listTables();
            if ((is_countable($tables) ? count($tables) : 0) > 0) {
                $this->conn->prepare("SET FOREIGN_KEY_CHECKS = 0;")-
>execute();

                foreach ($tables as $table) {
                    $sql = 'DROP TABLE ' .
MySQLDictionarySchemaGenerator::quoteString($table->getName());
                    $this->conn->prepare($sql)->execute();
                }
                $this->conn->prepare("SET FOREIGN_KEY_CHECKS = 1;")-
>execute();
            }
        } catch (\Exception $e) {
            $strMsg = "Failed deleting tables from database: " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
    }
```

To :

```
private function cleanDictionarySchema() {
        try {
            $tables = $this->conn->getSchemaManager()->listTables();
            // var_dump(count($tables)); die;
            if (count($tables) > 0) {
```

```
                $dictionaryLabel = str_replace(" ", "_", str_replace("_DICT",
"", $this->dictionary->getName()));
                $mystring = strtolower($dictionaryLabel)."_";
                foreach ($tables as $table) {
                    if(substr($table->getName(), 0, strlen($mystring)) ===
$mystring){

                        $sql = 'DROP TABLE "' . $table->getName() .'"
CASCADE';
                        // $this->conn->prepare('ALTER TABLE "' . $table-
>getName().'" DISABLE TRIGGER ALL;')->execute();
                        $this->conn->prepare($sql)->execute();
                        // $this->conn->prepare('ALTER TABLE "' . $table-
>getName().'" ENABLE TRIGGER ALL;')->execute();

                    }
                }
            }
        } catch (\Exception $e) {
            $strMsg = "Failed deleting tables from database: " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, array("context" => (string) $e));
            throw $e;
        }
    }
```

✓ Transform « createDictionarySchema » function :

From :

```
    private function createDictionarySchema($processCasesOptions) {

        $bind = [];
        try {
            $dictionarySchema = new MySQLDictionarySchemaGenerator($this-
>logger);
            $processCasesOptions = $this->getProcessCaseOptions();
            $schema = $dictionarySchema->generateDictionary($this->dictionary,
$processCasesOptions);
            $dictionarySQL = $schema->toSql($this->conn-
>getDatabasePlatform());
            $dictionarySQL = implode(";" . PHP_EOL, $dictionarySQL);
            $this->logger->debug("writing schema SQL " . $dictionarySQL);

            $this->conn->prepare($dictionarySQL)->execute();

            //insert into cspro_meta dictionary information
```

```
            $dictionaryVersion = $this->dictionary->getVersion();
            $stm = "SELECT modified_time, `dictionary_full_content` FROM
`cspro_dictionaries` "
                    . " WHERE  `dictionary_name` = '" . $this->dictionaryName
. "'";
            $result = $this->pdo->fetchOne($stm);
            if ($result) {
                $stm = "INSERT INTO `cspro_meta`(`cspro_version`,
`dictionary`, `source_modified_time`) "
                        . "VALUES (:version, :dictionary,
:source_modified_time)";
                $bind['version'] = $dictionaryVersion;
                $bind['dictionary'] = $result['dictionary_full_content'];
                $bind['source_modified_time'] = $result['modified_time'];
                $stmt = $this->conn->executeUpdate($stm, $bind);
            }
        } catch (\Exception $e) {
            $strMsg = "Failed generating tables in database: " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
    }
```

To :

```
    private function createDictionarySchema($processCasesOptions) {

        $bind = [];
        try {
            $dictionarySchema = new MySQLDictionarySchemaGenerator($this-
>logger);
            $processCasesOptions = $this->getProcessCaseOptions();
            $schema = $dictionarySchema->generateDictionary($this->dictionary,
$processCasesOptions);
            $dictionarySQL = $schema->toSql($this->conn-
>getDatabasePlatform());
            $explodedDictionarySQL = implode(";" . PHP_EOL, $dictionarySQL);
            $this->logger->debug("writing schema SQL " .
$explodedDictionarySQL);

            foreach($dictionarySQL AS $oneDictionarySQL){
                $this->conn->prepare($oneDictionarySQL)->execute();
            }

            //insert into cspro_meta dictionary information
```

```
            $dictionaryVersion = $this->dictionary->getVersion();

            //Récupération du label du dictionnaire concerné par les
insertions dans la table labeldictionnaire_cspro_meta
            $dictionaryLabel = str_replace(" ", "_", str_replace("_DICT", "",
$this->dictionary->getName()));

            $stm = "SELECT modified_time, `dictionary_full_content` FROM
`cspro_dictionaries` "
                    . " WHERE  `dictionary_name` = '" . $this->dictionaryName
. "'";
            $result = $this->pdo->fetchOne($stm);
            if ($result) {
                $stm = "INSERT INTO
".strtolower($dictionaryLabel)."_cspro_meta(cspro_version, dictionary,
source_modified_time) "
                . "VALUES (:version, :dictionary, :source_modified_time)";
                $bind['version'] = $dictionaryVersion;
                $bind['dictionary'] = $result['dictionary_full_content'];
                $bind['source_modified_time'] = $result['modified_time'];
                $stmt = $this->conn->executeUpdate($stm, $bind);
            }
        } catch (\Exception $e) {
            $strMsg = "Failed generating tables in database: " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
    }
```

- ✓ Transform « generateDictionary » function :

From :

```
    public function generateDictionary(Dictionary $dictionary,
$processCasesOptions) {

        DictionarySchemaHelper::updateProcessCasesOptions($dictionary,
$processCasesOptions);
        $this->schema = new Schema();
        $this->createDefaultTables();
        $parentLevel = null;
        //TODO check for Charset | Collation | comment
        for ($iLevel = 0; $iLevel < (is_countable($dictionary->getLevels()) ?
count($dictionary->getLevels()) : 0); $iLevel++) {
            $level = $dictionary->getLevels()[$iLevel];
            $level->setLevelNumber($iLevel);
            $this->generateLevel($level, $parentLevel);
```

```
            $parentLevel = $dictionary->getLevels()[$iLevel];
        }
        return $this->schema;
    }
```

To :

```
    public function generateDictionary(Dictionary $dictionary,
$processCasesOptions) {

        //Récupérer le label du dictionnaire sur lequel on se trouve pour la
synchronisation des données
               $this->nomSchama = str_replace(" ", "_",
strtolower(str_replace("_DICT", "", $dictionary->getName())))); ;



        DictionarySchemaHelper::updateProcessCasesOptions($dictionary,
$processCasesOptions);
        $this->schema = new Schema();
        $this->createDefaultTables();
        $parentLevel = null;
        //TODO check for Charset | Collation | comment
        for ($iLevel = 0; $iLevel < (is_countable($dictionary->getLevels()) ?
count($dictionary->getLevels()) : 0); $iLevel++) {
            $level = $dictionary->getLevels()[$iLevel];
            $level->setLevelNumber($iLevel);
            $this->generateLevel($level, $parentLevel);
            $parentLevel = $dictionary->getLevels()[$iLevel];
        }
        return $this->schema;
    }
```

NB : Dont forget to initialize « nomSchema » in **_construct** function.

- ✓ Transform « createDefaultTables » function :

From :

```
    public function createDefaultTables() {
//cases
        /* "CREATE TABLE cases ("
          "id TEXT NOT NULL,"
          "`key` TEXT NOT NULL,"
          "label TEXT,"
          "questionnaire TEXT NOT NULL,"
          "last_modified_revision INTEGER NOT NULL,"
          "deleted INTEGER NOT NULL DEFAULT 0,"
          "verified INTEGER NOT NULL DEFAULT 0,"
```

```
          "partial_save_mode TEXT NULL,"
          "partial_save_field_name TEXT NULL,"
          "partial_save_level_key TEXT NULL,"
          "partial_save_record_occurrence INTEGER NULL,"
          "partial_save_item_occurrence INTEGER NULL,"
          "partial_save_subitem_occurrence INTEGER NULL,"
          "FOREIGN KEY(last_modified_revision) REFERENCES file_revisions(id)"
          ");\n" */
        $casesTable = $this->schema-
>createTable(static::quoteString('cases'));
        $casesTable->addColumn(static::quoteString('id'), "text", ["notnull"
=> true]);
        $casesTable->addColumn(static::quoteString('key'), "text", ["notnull"
=> true]);
        $casesTable->addColumn(static::quoteString('label'), "text");
        $casesTable->addColumn(static::quoteString('questionnaire'), "text",
["notnull" => false, "default" => null]);
        $casesTable->addColumn(static::quoteString('last_modified_revision'),
"integer", ["notnull" => true]);
        $casesTable->addColumn(static::quoteString('deleted'), "integer",
["notnull" => true, "default" => 0]);
        $casesTable->addColumn(static::quoteString('verified'), "integer",
["notnull" => true, "default" => 0]);
        $casesTable->addColumn(static::quoteString('partial_save_mode'),
"text", ["notnull" => false, "default" => null]);
        $casesTable->addColumn(static::quoteString('partial_save_field_name'),
"text", ["notnull" => false, "default" => null]);
        $casesTable->addColumn(static::quoteString('partial_save_level_key'),
"text", ["notnull" => false, "default" => null]);
        $casesTable-
>addColumn(static::quoteString('partial_save_record_occurrence'), "integer",
["notnull" => false, "default" => null]);
        $casesTable-
>addColumn(static::quoteString('partial_save_item_occurrence'), "integer",
["notnull" => false, "default" => null]);
        $casesTable-
>addColumn(static::quoteString('partial_save_subitem_occurrence'), "integer",
["notnull" => false, "default" => null]);

        $casesTable->addUniqueIndex(["`id`"], null, ["lengths" => [191]]);
        $casesTable->addIndex([static::quoteString('deleted')]);
//notes
        /*     "CREATE TABLE notes ("
          "case_id TEXT NOT NULL,"
          "field_name TEXT NOT NULL,"
          "level_key TEXT NOT NULL,"
          "record_occurrence INTEGER NOT NULL,"
          "item_occurrence INTEGER NOT NULL,"
          "subitem_occurrence INTEGER NOT NULL,"
```

```
          "content TEXT NOT NULL,"
          "operator_id TEXT NOT NULL,"
          "modified_time INTEGER NOT NULL,"
          "FOREIGN KEY(case_id) REFERENCES cases(id)"
          ");\n"
          "CREATE INDEX `notes-case-id` ON notes(case_id);"; */
        $notesTable = $this->schema-
>createTable(static::quoteString('notes'));
        $notesTable->addColumn(static::quoteString('case_id'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('field_name'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('level_key'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('record_occurrence'),
"integer", ["notnull" => true]);
        $notesTable->addColumn(static::quoteString('item_occurrence'),
"integer", ["notnull" => true]);
        $notesTable->addColumn(static::quoteString('subitem_occurrence'),
"integer", ["notnull" => true]);
        $notesTable->addColumn(static::quoteString('content'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('operator_id'), "text",
["notnull" => true]);
        $notesTable->addColumn("`modified_time`", "datetime",
['columnDefinition' => 'timestamp']);
        $notesTable->addIndex([static::quoteString('case_id')], null, [],
["lengths" => [191]]);
        //DBAL has issues with creating foreign key constraint on text columns
with lengths.
        //not adding for now, if needed add it in the future
//        $notesTable->addForeignKeyConstraint($this->quoteString('cases'),
array($this->quoteString('case_id')), array($this->quoteString('id')),
//                array("lengths" => array(191,191)), 'notes_cases_fk');

        /* CREATE TABLE IF NOT EXISTS `cspro_jobs` (
          `id` int(11) unsigned NOT NULL AUTO_INCREMENT,
          `startCaseId` int unsigned NOT NULL,
          `startRevision` int unsigned NOT NULL,
          `endCaseId` int unsigned NOT NULL,
          `endRevision` int unsigned NOT NULL,
          `casesProcessed` int unsigned  NULL,
          `created_time` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
          `modified_time` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP,
          PRIMARY KEY (`id`)
          ) ENGINE=InnoDB  DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
*/
```

```
        $jobsTable = $this->schema-
>createTable(static::quoteString('cspro_jobs'));
        $jobsTable->addColumn("`id`", "integer", ["unsigned" => true,
"notnull" => true, "autoincrement" => true]);
        $jobsTable->addColumn("`start_caseid`", "integer", ["unsigned" =>
true, "notnull" => true]);
        $jobsTable->addColumn("`start_revision`", "integer", ["unsigned" =>
true, "notnull" => true]);
        $jobsTable->addColumn("`end_caseid`", "integer", ["unsigned" => true,
"notnull" => true]);
        $jobsTable->addColumn("`end_revision`", "integer", ["unsigned" =>
true, "notnull" => true]);
        $jobsTable->addColumn("`cases_to_process`", "integer", ["unsigned" =>
true, "notnull" => false, "default" => null]);
        $jobsTable->addColumn("`cases_processed`", "integer", ["unsigned" =>
true, "notnull" => false, "default" => null]);
        $jobsTable->addColumn("`status`", "integer", ["unsigned" => true,
"notnull" => true, "default" => 0]);
        $jobsTable->addColumn("`created_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp']);
        $jobsTable->addColumn("`modified_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp on update
current_timestamp']);
        $jobsTable->setPrimaryKey(["`id`"]);

        //Create meta table
        $metaTable = $this->schema-
>createTable(static::quoteString('cspro_meta'));
        $metaTable->addColumn("`id`", "integer", ["unsigned" => true,
"notnull" => true, "autoincrement" => true]);
        $metaTable->addColumn("`cspro_version`", "text", ["notnull" => true]);
        $metaTable->addColumn("`dictionary`", "text", ["notnull" => true]);
        $metaTable->addColumn("`source_modified_time`", "datetime", ["default"
=> null]);
        $metaTable->addColumn("`created_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp']);
        $metaTable->addColumn("`modified_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp on update
current_timestamp']);
        $metaTable->setPrimaryKey(["`id`"]);
    }
```

To :

```
    public function createDefaultTables()
    {
        //cases
        /* "CREATE TABLE cases ("
```

```
                "id TEXT NOT NULL,"
                "`key` TEXT NOT NULL,"
                "label TEXT,"
                "questionnaire TEXT NOT NULL,"
                "last_modified_revision INTEGER NOT NULL,"
                "deleted INTEGER NOT NULL DEFAULT 0,"
                "verified INTEGER NOT NULL DEFAULT 0,"
                "partial_save_mode TEXT NULL,"
                "partial_save_field_name TEXT NULL,"
                "partial_save_level_key TEXT NULL,"
                "partial_save_record_occurrence INTEGER NULL,"
                "partial_save_item_occurrence INTEGER NULL,"
                "partial_save_subitem_occurrence INTEGER NULL,"
                "FOREIGN KEY(last_modified_revision) REFERENCES
file_revisions(id)"
                ");\n" */
        $casesTable = $this->schema->createTable(static::quoteString($this-
>nomSchama.'_cases'));
        $casesTable->addColumn(static::quoteString('id'), "text", ["notnull"
=> true]);
        $casesTable->addColumn(static::quoteString('key'), "text", ["notnull"
=> true]);
        $casesTable->addColumn(static::quoteString('label'), "text");
        $casesTable->addColumn(static::quoteString('questionnaire'), "text",
["notnull" => false, "default" => null]);
        $casesTable->addColumn(static::quoteString('last_modified_revision'),
"integer", ["notnull" => true]);
        $casesTable->addColumn(static::quoteString('deleted'), "integer",
["notnull" => true, "default" => 0]);
        $casesTable->addColumn(static::quoteString('verified'), "integer",
["notnull" => true, "default" => 0]);
        $casesTable->addColumn(static::quoteString('partial_save_mode'),
"text", ["notnull" => false, "default" => null]);
        $casesTable->addColumn(static::quoteString('partial_save_field_name'),
"text", ["notnull" => false, "default" => null]);
        $casesTable->addColumn(static::quoteString('partial_save_level_key'),
"text", ["notnull" => false, "default" => null]);
        $casesTable-
>addColumn(static::quoteString('partial_save_record_occurrence'), "integer",
["notnull" => false, "default" => null]);
        $casesTable-
>addColumn(static::quoteString('partial_save_item_occurrence'), "integer",
["notnull" => false, "default" => null]);
        $casesTable-
>addColumn(static::quoteString('partial_save_subitem_occurrence'), "integer",
["notnull" => false, "default" => null]);

        $casesTable->addUniqueIndex(["`id`"], null, ["lengths" => [191]]);
        $casesTable->addIndex([static::quoteString('deleted')]);
```

```
        //notes
        /*     "CREATE TABLE notes ("
              "case_id TEXT NOT NULL,"
              "field_name TEXT NOT NULL,"
              "level_key TEXT NOT NULL,"
              "record_occurrence INTEGER NOT NULL,"
              "item_occurrence INTEGER NOT NULL,"
              "subitem_occurrence INTEGER NOT NULL,"
              "content TEXT NOT NULL,"
              "operator_id TEXT NOT NULL,"
              "modified_time INTEGER NOT NULL,"
              "FOREIGN KEY(case_id) REFERENCES cases(id)"
              ");\n"
              "CREATE INDEX `notes-case-id` ON notes(case_id);"; */
        $notesTable = $this->schema->createTable(static::quoteString($this-
>nomSchama.'_notes'));
        $notesTable->addColumn(static::quoteString('case_id'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('field_name'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('level_key'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('record_occurrence'),
"integer", ["notnull" => true]);
        $notesTable->addColumn(static::quoteString('item_occurrence'),
"integer", ["notnull" => true]);
        $notesTable->addColumn(static::quoteString('subitem_occurrence'),
"integer", ["notnull" => true]);
        $notesTable->addColumn(static::quoteString('content'), "text",
["notnull" => true]);
        $notesTable->addColumn(static::quoteString('operator_id'), "text",
["notnull" => true]);
        $notesTable->addColumn("`modified_time`", "datetime",
['columnDefinition' => 'timestamp']);
        $notesTable->addIndex([static::quoteString('case_id')], null, [],
["lengths" => [191]]);
        //DBAL has issues with creating foreign key constraint on text columns
with lengths.
        //not adding for now, if needed add it in the future
        //        $notesTable->addForeignKeyConstraint($this-
>quoteString('cases'), array($this->quoteString('case_id')), array($this-
>quoteString('id')),
        //                array("lengths" => array(191,191)),
'notes_cases_fk');

        /* CREATE TABLE IF NOT EXISTS `cspro_jobs` (
              `id` int(11) unsigned NOT NULL AUTO_INCREMENT,
              `startCaseId` int unsigned NOT NULL,
              `startRevision` int unsigned NOT NULL,
```

```
              `endCaseId` int unsigned NOT NULL,
              `endRevision` int unsigned NOT NULL,
              `casesProcessed` int unsigned  NULL,
              `created_time` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
              `modified_time` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP,
              PRIMARY KEY (`id`)
              ) ENGINE=InnoDB  DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci; */
        $jobsTable = $this->schema->createTable(static::quoteString($this-
>nomSchama.'_cspro_jobs'));
        $jobsTable->addColumn("`id`", "integer", ["unsigned" => true,
"notnull" => true, "autoincrement" => true]);
        $jobsTable->addColumn("`start_caseid`", "integer", ["unsigned" =>
true, "notnull" => true]);
        $jobsTable->addColumn("`start_revision`", "integer", ["unsigned" =>
true, "notnull" => true]);
        $jobsTable->addColumn("`end_caseid`", "integer", ["unsigned" => true,
"notnull" => true]);
        $jobsTable->addColumn("`end_revision`", "integer", ["unsigned" =>
true, "notnull" => true]);
        $jobsTable->addColumn("`cases_to_process`", "integer", ["unsigned" =>
true, "notnull" => false, "default" => null]);
        $jobsTable->addColumn("`cases_processed`", "integer", ["unsigned" =>
true, "notnull" => false, "default" => null]);
        $jobsTable->addColumn("`status`", "integer", ["unsigned" => true,
"notnull" => true, "default" => 0]);
        $jobsTable->addColumn("`created_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp']);
        $jobsTable->addColumn("`modified_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp']);
        $jobsTable->setPrimaryKey(["`id`"]);

        //Create meta table
        $metaTable = $this->schema->createTable(static::quoteString($this-
>nomSchama.'_cspro_meta'));
        $metaTable->addColumn("`id`", "integer", ["unsigned" => true,
"notnull" => true, "autoincrement" => true]);
        $metaTable->addColumn("`cspro_version`", "text", ["notnull" => true]);
        $metaTable->addColumn("`dictionary`", "text", ["notnull" => true]);
        $metaTable->addColumn("`source_modified_time`", "datetime", ["default"
=> null]);
        $metaTable->addColumn("`created_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp']);
        $metaTable->addColumn("`modified_time`", "datetime",
['columnDefinition' => 'timestamp default current_timestamp']);
        $metaTable->setPrimaryKey(["`id`"]);
    }
```

✓ Transform « generateLevelIdsTable » function :

From :

```
public function generateLevelIdsTable(Level $level, ?Level $parentLevel) {

        $levelName = "level-" . (string) ($level->getLevelNumber() + 1);
        $parentLevelName = $parentLevel ? "level-" . $parentLevel-
>getLevelNumber() : "case";
        $parentId = $parentLevelName . "-id";

        //create a table using DBAL
        $levelIdTable = $this->schema-
>createTable(static::quoteString($levelName));
        //add columns
        $autoIncrementFlag = $parentLevel ? false : true;
        $levelIdTable->addColumn(static::quoteString($levelName) . "-id",
"integer", ["unsigned" => true, "notnull" => true, "autoincrement" =>
$autoIncrementFlag]);
        // First level is linked to uuid in cases table, others are linked by
integer to parent level
        $parentKeyType = $parentLevel ? self::COLUMN_TYPE_INTEGER :
self::COLUMN_TYPE_TEXT;
        $unsignedFlag = $parentKeyType === self::COLUMN_TYPE_INTEGER ? true :
false;
        $levelIdTable->addColumn(static::quoteString($parentId),
$parentKeyType, ["unsigned" => $unsignedFlag, "notnull" => true]);

        // For second and third level nodes need an order column
        if ($level->getLevelNumber() > 0) {
            $levelIdTable->addColumn(static::quoteString("occ"), "integer",
["unsigned" => true, "notnull" => true, "default" => 1]);
        }
        //set primary key on id
        $levelIdTable->setPrimaryKey([static::quoteString($levelName . "-
id")]);

        //add id items to the levelId table
        for ($iItem = 0; $iItem < (is_countable($level->getIdItems()) ?
count($level->getIdItems()) : 0); $iItem++) {
            $this->addRecordItemToTable($levelIdTable, $level-
>getIdItems()[$iItem]);
        }

        if ($parentLevel) {
            $levelIdTable-
>addForeignKeyConstraint(static::quoteString($parentLevelName),
[static::quoteString($parentId)], [static::quoteString($parentId)],
["onDelete" => "CASCADE"]);
```

```
        }
        // Only first level parent id is unique since that is case.id, at
higher levels can have multiple child nodes
        $unique = $parentLevel == null ? true : false;

        if ($unique) {
            if ($parentKeyType === self::COLUMN_TYPE_TEXT) {
                $levelIdTable-
>addUniqueIndex([static::quoteString($parentId)], null, ["lengths" => [191]]);
            } else {
                $levelIdTable-
>addUniqueIndex([static::quoteString($parentId)]);
            }
        }
    }
```

To :

```
 public function generateLevelIdsTable(Level $level, ?Level $parentLevel) {

        $levelName = "level-" . (string) ($level->getLevelNumber() + 1);
        $parentLevelName = $parentLevel ? "level-" . $parentLevel-
>getLevelNumber() : "case";
        $parentId = $parentLevelName . "-id";

        //create a table using DBAL
        $levelIdTable = $this->schema->createTable(static::quoteString($this-
>nomSchama.'_'.$levelName));
        //add columns
        $autoIncrementFlag = $parentLevel ? false : true;
        $levelIdTable->addColumn(static::quoteString($levelName) . "-id",
"integer", ["unsigned" => true, "notnull" => true, "autoincrement" =>
$autoIncrementFlag]);
        // First level is linked to uuid in cases table, others are linked by
integer to parent level
        $parentKeyType = $parentLevel ? self::COLUMN_TYPE_INTEGER :
self::COLUMN_TYPE_TEXT;
        $unsignedFlag = $parentKeyType === self::COLUMN_TYPE_INTEGER ? true :
false;
        $levelIdTable->addColumn(static::quoteString($parentId),
$parentKeyType, ["unsigned" => $unsignedFlag, "notnull" => true]);

        // For second and third level nodes need an order column
        if ($level->getLevelNumber() > 0) {
            $levelIdTable->addColumn(static::quoteString("occ"), "integer",
["unsigned" => true, "notnull" => true, "default" => 1]);
        }
        //set primary key on id
```

```
        $levelIdTable->setPrimaryKey([static::quoteString($levelName . "-
id")]);

        //add id items to the levelId table
        for ($iItem = 0; $iItem < (is_countable($level->getIdItems()) ?
count($level->getIdItems()) : 0); $iItem++) {
            $this->addRecordItemToTable($levelIdTable, $level-
>getIdItems()[$iItem]);
        }

        if ($parentLevel) {
            $levelIdTable->addForeignKeyConstraint(static::quoteString($this-
>nomSchama.'_'.$parentLevelName), [static::quoteString($parentId)],
[static::quoteString($parentId)], ["onDelete" => "CASCADE"]);
        }
        // Only first level parent id is unique since that is case.id, at
higher levels can have multiple child nodes
        $unique = $parentLevel == null ? true : false;

        if ($unique) {
            if ($parentKeyType === self::COLUMN_TYPE_TEXT) {
                $levelIdTable-
>addUniqueIndex([static::quoteString($parentId)], null, ["lengths" => [191]]);
            } else {
                $levelIdTable-
>addUniqueIndex([static::quoteString($parentId)]);
            }
        }
    }
```

- ✓ Transform « generateRecordTable » function :

From :

```
public function generateRecordTable(Record $record) {
        $parentLevelName = "level-" . (string) ($record->getLevel()-
>getLevelNumber() + 1);
        $parentId = $parentLevelName . "-id";

        //create a table using DBAL
        $recordTable = $this->schema->createTable(strtolower($record-
>getName()));
        //add columns -added auto increment for MySQL for record-ids
        $recordTable->addColumn(static::quoteString(strtolower($record-
>getName()) . "-id"), "integer", ["unsigned" => true, "notnull" => true,
"autoincrement" => true]);
        //set primary key on id
```

```
        $recordTable->setPrimaryKey([static::quoteString(strtolower($record-
>getName()) . "-id")]);

        $recordTable->addColumn(static::quoteString($parentId), "integer",
["unsigned" => true, "notnull" => true]);

        //add occ column if max occs > 1
        if ($record->getMaxRecords() > 1) {
            $recordTable->addColumn(static::quoteString("occ"), "integer",
["unsigned" => true, "notnull" => true, "default" => 1]);
        }

        $this->addRecordItemsToTable($recordTable, $record);

        $recordTable-
>addForeignKeyConstraint(static::quoteString($parentLevelName),
[static::quoteString($parentId)], [static::quoteString($parentId)],
["onDelete" => "CASCADE"]);
        $recordTable->addIndex([static::quoteString($parentId)]);
    }
```

To :

```
 public function generateRecordTable(Record $record) {
        $parentLevelName = "level-" . (string) ($record->getLevel()-
>getLevelNumber() + 1);
        $parentId = $parentLevelName . "-id";

        //create a table using DBAL
        $recordTable = $this->schema->createTable(strtolower($this-
>nomSchama.'_'.$record->getName()));
        //add columns -added auto increment for MySQL for record-ids
        $recordTable->addColumn(static::quoteString(strtolower($record-
>getName()) . "-id"), "integer", ["unsigned" => true, "notnull" => true,
"autoincrement" => true]);
        //set primary key on id
        $recordTable->setPrimaryKey([static::quoteString(strtolower($record-
>getName()) . "-id")]);

        $recordTable->addColumn(static::quoteString($parentId), "integer",
["unsigned" => true, "notnull" => true]);

        //add occ column if max occs > 1
        if ($record->getMaxRecords() > 1) {
            $recordTable->addColumn(static::quoteString("occ"), "integer",
["unsigned" => true, "notnull" => true, "default" => 1]);
        }
```

```
        $this->addRecordItemsToTable($recordTable, $record);

        $recordTable->addForeignKeyConstraint(static::quoteString($this-
>nomSchama.'_'.$parentLevelName), [static::quoteString($parentId)],
[static::quoteString($parentId)], ["onDelete" => "CASCADE"]);
        $recordTable->addIndex([static::quoteString($parentId)]);
    }
```

- ✓ Transform « IsValidSchema » function :

From :

```
public function IsValidSchema(): bool {
        $bind = [];
        //check the time stamp of dictionary in the meta table with the
original dictionary timestamp.
        $isValid = false;
        try {
            if (!$this->tableExists("`cspro_meta`")) {
                return $isValid;
            }
            $stm = "SELECT source_modified_time FROM `cspro_meta` ";
            $stmt = $this->conn->executeQuery($stm);
            $result = $stmt->fetch();
            if ($result) {
                $stm = "SELECT count(*) FROM `cspro_dictionaries` "
                        . " WHERE  `dictionary_name` = :dictionaryName and
`modified_time` = :source_modified_time";
                $bind['dictionaryName'] = $this->dictionaryName;
                $bind['source_modified_time'] =
$result['source_modified_time'];

                $result = (int) $this->pdo->fetchValue($stm, $bind);
                $isValid = ($result === 1) ? true : false;
            }
        } catch (\Exception $e) {
            $strMsg = "Failed validating schema  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
        $this->logger->debug('The schema valid flag is ' . $isValid);
        return $isValid;
    }
```

To :

```
    public function IsValidSchema(): bool {
        $bind = [];
        //check the time stamp of dictionary in the meta table with the
original dictionary timestamp.
        $isValid = false;

        //Récupération du label du dictionnaire concerné par les insertions
dans la table labeldictionnaire_cspro_meta
        $dictionaryLabel = str_replace(" ", "_", str_replace("_DICT", "",
$this->dictionary->getName()));

        try {
            if (!$this->tableExists(
"".strtolower($dictionaryLabel)."_cspro_meta")) {
                return $isValid;
            }
            $stm = "SELECT source_modified_time FROM
".strtolower($dictionaryLabel)."_cspro_meta ";
            $stmt = $this->conn->executeQuery($stm);
            $result = $stmt->fetch();
            if ($result) {
                $stm = "SELECT count(*) FROM `cspro_dictionaries` "
                        . " WHERE  `dictionary_name` = :dictionaryName and
`modified_time` = :source_modified_time";
                $bind['dictionaryName'] = $this->dictionaryName;
                $bind['source_modified_time'] =
$result['source_modified_time'];

                $result = (int) $this->pdo->fetchValue($stm, $bind);
                $isValid = ($result === 1) ? true : false;
            }
        } catch (\Exception $e) {
            $strMsg = "Failed validating schema  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
        $this->logger->debug('The schema valid flag is ' . $isValid);
        return $isValid;
    }
```

- ✓ Transform « resetInProcessJob » function :

From :

```
    public function resetInProcesssJobs(): int {
        $bind = [];
```

```
        try {
            $stm = "UPDATE `cspro_jobs` SET `status`= :status WHERE `status` =
:in_process_jobs";
            $bind['status'] = self::JOB_STATUS_NOT_STARTED;
            $bind['in_process_jobs'] = self::JOB_STATUS_IN_PROCESS;
            $count = $this->conn->executeUpdate($stm, $bind);
        } catch (\Exception $e) {
            $strMsg = "Failed resetting jobs in schema  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
        return $count;
    }
```

To :

```
    public function resetInProcesssJobs(): int {
        $bind = [];
        //Récupération du label du dictionnaire concerné par les insertions
dans la table labeldictionnaire_cspro_jobs
        $dictionaryLabel = str_replace(" ", "_", str_replace("_DICT", "",
$this->dictionary->getName()));

        try {
            $stm = "UPDATE ".strtolower($dictionaryLabel)."_cspro_jobs SET
status= :status WHERE status = :in_process_jobs";
            $bind['status'] = self::JOB_STATUS_NOT_STARTED;
            $bind['in_process_jobs'] = self::JOB_STATUS_IN_PROCESS;
            $count = $this->conn->executeUpdate($stm, $bind);
        } catch (\Exception $e) {
            $strMsg = "Failed resetting jobs in schema  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
        return $count;
    }
```

- ✓ Transform « ProcessNextJob » function :

From :

```
    public function processNextJob($maxCasesPerChunk): int {
```

```
        $bind = [];
        $jobId = 0;
//find a job that is not being processed and update its status to processing
        try {
            $stm = "SELECT `id` FROM `cspro_jobs` "
                    . " WHERE  `status` = " . self::JOB_STATUS_NOT_STARTED . "
ORDER BY `id`  LIMIT 1 ";
            $stmt = $this->conn->prepare($stm);
            $resultSet = $stmt->execute();
            $result = $resultSet->fetchAllAssociative();
            $jobId = (is_countable($result) ? count($result) : 0) > 0 ?
$result[0]['id'] : $this->createJob($maxCasesPerChunk);
            if ($jobId) {
                $stm = "UPDATE `cspro_jobs` SET `status`= :status WHERE `id` =
:id";
                $bind['status'] = self::JOB_STATUS_IN_PROCESS;
                $bind['id'] = $jobId;
                $this->conn->executeUpdate($stm, $bind);
            }
        } catch (\Exception $e) {
            $strMsg = "Failed getting next job from database:  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
        return $jobId;
    }
```

To :

```
public function processNextJob($maxCasesPerChunk): int {
        //Récupération du label du dictionnaire concerné par les insertions
dans la table labeldictionnaire_cspro_jobs
        $dictionaryLabel = str_replace(" ", "_", str_replace("_DICT", "",
$this->dictionary->getName()));

        $bind = [];
        $jobId = 0;
//find a job that is not being processed and update its status to processing
        try {
            $stm = "SELECT id FROM ".strtolower($dictionaryLabel)."_cspro_jobs
"
            . " WHERE  status = " . self::JOB_STATUS_NOT_STARTED . " ORDER BY
id  LIMIT 1 ";

            $stmt = $this->conn->prepare($stm);
            $resultSet = $stmt->execute();
```

```
            $result = $resultSet->fetchAllAssociative();
            $jobId = (is_countable($result) ? count($result) : 0) > 0 ?
$result[0]['id'] : $this->createJob($maxCasesPerChunk);
            if ($jobId) {
                $stm = "UPDATE ".strtolower($dictionaryLabel)."_cspro_jobs SET
status= :status WHERE id = :id";
                $bind['status'] = self::JOB_STATUS_IN_PROCESS;
                $bind['id'] = $jobId;
                $this->conn->executeUpdate($stm, $bind);
            }
        } catch (\Exception $e) {
            $strMsg = "Failed getting next job from database:  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
        return $jobId;
    }
```

- ✓ Transform « createJob » function :

From :

```
    public function createJob($maxCasesPerChunk): int {
        $bind = [];
        //if a job already exists - get the endCaseId and endRevision if there
are no cases at this revision
//SELECT the most recent job and get the endCaseId and endRevision
        $jobId = 0;
        $stm = "SELECT `id`, `start_caseid`, `start_revision`, `end_caseid`,
`end_revision`, `cases_processed`, `status` FROM `cspro_jobs` "
                . "ORDER BY `id` DESC LIMIT 1 ";

        try {
            $stmt = $this->conn->prepare($stm);
            $resultSet = $stmt->execute();
            $result = $resultSet->fetchAllAssociative();
            $endRevision = 0;
            $endCaseId = 0;
            if ($result) {
                $endRevision = $result[0]['end_revision'];
                $endCaseId = $result[0]['end_caseid'];
            }
//select cases from the source cases  table  where revision =
end_revision  end_revision id > end_caseid
            $stm = "SELECT `id`, `revision` FROM " . $this->dictionaryName . "
WHERE revision = :endRevision and `id` > :endCaseId  "
```

```
                    . " UNION "
                    . "SELECT `id`, `revision` FROM " . $this->dictionaryName
. " WHERE revision > :endRevision "
                    . "ORDER BY `revision`, `id` LIMIT :limit";

            $limit = $maxCasesPerChunk;
            $stmt = $this->pdo->prepare($stm);
            $stmt->bindParam(':limit', $limit, \PDO::PARAM_INT);
            $stmt->bindParam(':endCaseId', $endCaseId, \PDO::PARAM_INT);
            $stmt->bindParam(':endRevision', $endRevision, \PDO::PARAM_INT);

            $stmt->execute();
            $result = $stmt->fetchAll();
            if ($result) {
                unset($bind);
                $bind['startCaseId'] = $result[0]['id'];
                $bind['startRevision'] = $result[0]['revision'];
                $bind['endCaseId'] = $result[count($result) - 1]['id'];
                $bind['endRevision'] = $result[count($result) -
1]['revision'];
                $bind['cases_to_process'] = count($result);
                $stm = "INSERT INTO `cspro_jobs`(`start_caseid`,
`start_revision`, `end_caseid`, `end_revision` ,`cases_to_process`) "
                        . "VALUES (:startCaseId, :startRevision, :endCaseId,
:endRevision, :cases_to_process)";
                $stmt = $this->conn->executeUpdate($stm, $bind);
                $jobId = $this->conn->lastInsertId();
            }
            return $jobId;
        } catch (\Exception $e) {
            $strMsg = "Failed creating job in database:  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
    }
```

To :

```
    public function createJob($maxCasesPerChunk): int {
        //Récupération du label du dictionnaire concerné par les requetes sur
la table labeldictionnaire_cspro_jobs
        $dictionaryLabel = str_replace(" ", "_", str_replace("_DICT", "",
$this->dictionary->getName()));

        $bind = [];
```

```
        //if a job already exists - get the endCaseId and endRevision if there
are no cases at this revision
        //SELECT the most recent job and get the endCaseId and endRevision
        $jobId = 0;

        $stm = "SELECT id, start_caseid, start_revision, end_caseid,
end_revision, cases_processed, status FROM
".strtolower($dictionaryLabel)."_cspro_jobs "
                . "ORDER BY id DESC LIMIT 1 ";
        try {
            $stmt = $this->conn->prepare($stm);
            $resultSet = $stmt->execute();
            $result = $resultSet->fetchAllAssociative();
            $endRevision = 0;
            $endCaseId = 0;
            if ($result) {
                $endRevision = $result[0]['end_revision'];
                $endCaseId = $result[0]['end_caseid'];
            }
        //select cases from the source cases  table  where revision =
end_revision  end_revision id > end_caseid
            $stm = "SELECT `id`, `revision` FROM " . $this->dictionaryName . "
WHERE revision = :endRevision and `id` > :endCaseId  "
                    . " UNION "
                    . "SELECT `id`, `revision` FROM " . $this->dictionaryName
. " WHERE revision > :endRevision "
                    . "ORDER BY `revision`, `id` LIMIT :limit";

            $limit = $maxCasesPerChunk;
            $stmt = $this->pdo->prepare($stm);
            $stmt->bindParam(':limit', $limit, \PDO::PARAM_INT);
            $stmt->bindParam(':endCaseId', $endCaseId, \PDO::PARAM_INT);
            $stmt->bindParam(':endRevision', $endRevision, \PDO::PARAM_INT);

            $stmt->execute();
            $result = $stmt->fetchAll();
            if ($result) {
                unset($bind);
                $bind['startCaseId'] = $result[0]['id'];
                $bind['startRevision'] = $result[0]['revision'];
                $bind['endCaseId'] = $result[count($result) - 1]['id'];
                $bind['endRevision'] = $result[count($result) -
1]['revision'];
                $bind['cases_to_process'] = count($result);
                $stm = "INSERT INTO
".strtolower($dictionaryLabel)."_cspro_jobs(start_caseid, start_revision,
end_caseid, end_revision ,cases_to_process) "
                . "VALUES (:startCaseId, :startRevision, :endCaseId,
:endRevision, :cases_to_process)";
```

```
                $stmt = $this->conn->executeUpdate($stm, $bind);
                $jobId = $this->conn->lastInsertId();
            }
            return $jobId;
        } catch (\Exception $e) {
            $strMsg = "Failed creating job in database:  " . $this-
>connectionParams['dbname'] . " while processsing Dictionary: " . $this-
>dictionaryName;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw $e;
        }
    }
```

✓ Transform « getDataCounts » function :

From :

```
    public function getDataCounts(&$dataSettings) {
        //get each dictionary get the counts in the source and target schema
        foreach ($dataSettings as &$dataSetting) {
            $dataSetting['totalCases'] = "";
            $dataSetting['processedCases'] = "";
            $dataSetting['lastProcessedTime'] = "";

            if (isset($dataSetting['targetSchemaName'])) {
                $stm = "SELECT count(*) FROM `" . $dataSetting['name'] . "`
WHERE `deleted` = 0";
                $caseCount = (int) $this->pdo->fetchValue($stm);
                $dataSetting['totalCases'] = $caseCount;

        //get number of cases processsed.
                $connectionParams = ['dbname' =>
$dataSetting['targetSchemaName'], 'user' => $dataSetting['dbUserName'],
'password' => $dataSetting['dbPassword'], 'host' =>
$dataSetting['targetHostName'], 'driver' => 'pdo_pgsql'];
                $config = new Configuration();
                try {
                    $conn = DriverManager::getConnection($connectionParams,
$config);

        //get processsed case count
                    $dataSetting['processedCases'] = 0;
                    $statement = $conn->executeQuery('SELECT count(*) FROM
`cases` where `deleted`=0');
                    $processedCases = $statement->fetchOne();
                    $dataSetting['processedCases'] = $processedCases;

        //get processed time (modified time) from the most recently processed
job
```

```
                    $statement = $conn->executeQuery('SELECT id ,
modified_time FROM `cspro_jobs` WHERE id = (SELECT max(id) from cspro_jobs
where status =2)');
                    if (($row = $statement->fetchAssociative()) !== false) {
                        $dataSetting['lastProcessedTime'] =
$row['modified_time'];
                    }
                } catch (\Exception $e) {
                    if (strpos((string) $e, 'SQLSTATE[42S02]') == FALSE) {
                        $this->logger->error('Failed getting case counts and
last processed time', ["context" => (string) $e]);
                    }
                }
            }
        }
        return $dataSettings;
    }
```

To :

```
    public function getDataCounts(&$dataSettings) {
        //get each dictionary get the counts in the source and target schema
        foreach ($dataSettings as &$dataSetting) {
            $dataSetting['totalCases'] = "";
            $dataSetting['processedCases'] = "";
            $dataSetting['lastProcessedTime'] = "";

            if (isset($dataSetting['targetSchemaName'])) {
                $name_dict = str_replace(" ", "_", str_replace("_DICT", "",
$dataSetting['name'])).."_";

                $stm = "SELECT count(*) FROM `" . $dataSetting['name'] . "`
WHERE `deleted` = 0";
                $caseCount = (int) $this->pdo->fetchValue($stm);
                $dataSetting['totalCases'] = $caseCount;

        //get number of cases processsed.
                $connectionParams = ['dbname' =>
$dataSetting['targetSchemaName'], 'user' => $dataSetting['dbUserName'],
'password' => $dataSetting['dbPassword'], 'host' =>
$dataSetting['targetHostName'], 'driver' => 'pdo_pgsql'];
                $config = new Configuration();
                try {
                    $conn = DriverManager::getConnection($connectionParams,
$config);

        //get processsed case count
                    $dataSetting['processedCases'] = 0;
```

```
                    $statement = $conn->executeQuery('SELECT count(*) FROM
'.$name_dict.'cases where deleted=0');
                    $processedCases = $statement->fetchOne();
                    $dataSetting['processedCases'] = $processedCases;

        //get processed time (modified time) from the most recently processed
job
                    $statement = $conn->executeQuery('SELECT id ,
modified_time FROM '.$name_dict.'cspro_jobs WHERE id = (SELECT max(id) from
'.$name_dict.'cspro_jobs where status =2)');
                    if (($row = $statement->fetchAssociative()) !== false) {
                        $dataSetting['lastProcessedTime'] =
$row['modified_time'];
                    }
                } catch (\Exception $e) {
                    if (strpos((string) $e, 'SQLSTATE[42S02]') == FALSE) {
                        $this->logger->error('Failed getting case counts and
last processed time', ["context" => (string) $e]);
                    }
                }
            }
        }
        return $dataSettings;
    }
```

✓ Transform « _contruct » function in class « MySQLQuestionnaireSerializer » :

From :

```
    public function __construct(private Dictionary $dict, private $jobId,
private PdoHelper $sourcePdo, private Connection $targetConnection, private
LoggerInterface $logger) {
        $this->casesMap = [];
    }
```

To :

```
    public function __construct(private Dictionary $dict, private $jobId,
private PdoHelper $sourcePdo, private Connection $targetConnection, private
LoggerInterface $logger) {
        $this->casesMap = [];
        // Récupération du label du dictionnaire pour mettre à jour le préfix
des tables de report
        $this->labelDictionnaire = str_replace(" ", "_", str_replace("_DICT",
"", $dict->getName()));
    }
```

Nb : Dont forget to declare the private variable « labelDictionnaire » before function.

- ✓ Transform « serializeQuestionnaries » function :

From :

```
    public function serializeQuestionnaries($processCasesOptions) {
        $bind = [];
        DictionarySchemaHelper::updateProcessCasesOptions($this->dict,
$processCasesOptions);
        //        ini_set('memory_limit', '16G'); //increase memory if php
memory limit is hit
        $this->getJobInformation();
        $this->getQuestionnarieListToSerilaize();
        if (count($this->casesMap) == 0) {
            $this->logger->warning("No cases available to serialize for jobId:
" . $this->jobId);
            return;
        }
        $strMsg = "Serializing " . count($this->casesMap) . " cases for
dictionary : " . $this->dict->getName();
        $this->logger->info($strMsg);
        //delete questionnaires in a separate transaction to avoid deadlock
issues
        try {
            $this->deleteQuestionnaires();
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed serializing cases';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }

        //begin transaction
        $this->targetConnection->beginTransaction();
        try {
            $caseCount = $this->serializeCases();
            $caseCount = $this->serializeQuestionnaireLevel();
            $this->serializeQuestionnaireRecords();
            $this->serializeNotes();

            //update job
            $jobId = $this->jobId;
            $stm = "UPDATE `cspro_jobs` SET `status`= :status,
`cases_processed` = :totalCases WHERE `id` = :jobId";
            $bind['status'] = DictionarySchemaHelper::JOB_STATUS_COMPLETE;
```

```
            $bind['jobId'] = $this->jobId;
            $bind['totalCases'] = $caseCount;
            $this->targetConnection->executeUpdate($stm, $bind);
            //commit
            $this->targetConnection->commit();
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed serializing cases';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            $this->targetConnection->rollBack();
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

To :

```
   public function serializeQuestionnaries($processCasesOptions) {
        $bind = [];
        DictionarySchemaHelper::updateProcessCasesOptions($this->dict,
$processCasesOptions);
        //        ini_set('memory_limit', '16G'); //increase memory if php
memory limit is hit
        $this->getJobInformation();
        $this->getQuestionnarieListToSerilaize();
        if (count($this->casesMap) == 0) {
            $this->logger->warning("No cases available to serialize for jobId:
" . $this->jobId);
            return;
        }
        $strMsg = "Serializing " . count($this->casesMap) . " cases for
dictionary : " . $this->dict->getName();
        $this->logger->info($strMsg);
        //delete questionnaires in a separate transaction to avoid deadlock
issues
        try {
            $this->deleteQuestionnaires();
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed serializing cases';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
```

```
        //begin transaction
        $this->targetConnection->beginTransaction();
        try {
            $caseCount = $this->serializeCases();
            $caseCount = $this->serializeQuestionnaireLevel();
            $this->serializeQuestionnaireRecords();
            $this->serializeNotes();

            //update job
            $jobId = $this->jobId;
            $stm = "UPDATE ".strtolower($this->labelDictionnaire)."_cspro_jobs
SET status= :status, cases_processed = :totalCases WHERE id = :jobId";
            $bind['status'] = DictionarySchemaHelper::JOB_STATUS_COMPLETE;
            $bind['jobId'] = $this->jobId;
            $bind['totalCases'] = $caseCount;
            $this->targetConnection->executeUpdate($stm, $bind);
            //commit
            $this->targetConnection->commit();
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed serializing cases';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            $this->targetConnection->rollBack();
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

- ✓ Transform « getJobInformation » function :

From :

```
public function getJobInformation() {
        try {
            $stm = "SELECT `id`, `start_caseid`, `start_revision`,
`end_caseid`, `end_revision`, `cases_to_process` FROM `cspro_jobs` "
                    . " WHERE  `id` = " . $this->jobId;
            $result = $this->targetConnection->fetchAllAssociative($stm);
            unset($this->job);
            if ($result) {
                $this->job = $result [0];
            }
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= "] Failed getting job information jobID: " . $this-
>jobId;
```

```
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

To :

```
    public function getJobInformation() {
        try {
            $stm = "SELECT id, start_caseid, start_revision, end_caseid,
end_revision, cases_to_process FROM ".strtolower($this-
>labelDictionnaire)."_cspro_jobs "
                    . " WHERE  id = " . $this->jobId;
            $result = $this->targetConnection->fetchAllAssociative($stm);
            unset($this->job);
            if ($result) {
                $this->job = $result [0];
            }
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= "] Failed getting job information jobID: " . $this-
>jobId;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

- ✓ Transform « deleteQuestionnaires » function :

From :

```
    public function deleteQuestionnaires() {
        //delete all the questionnaires that match
        $caseList = array_keys($this->casesMap);
        $strCaseList = "'" . implode("','", $caseList) . "'";

        $this->targetConnection->beginTransaction();
        try {
            //delete existing cases
            $stm = 'DELETE FROM `cases` WHERE `id` in ( ' . $strCaseList .
")";
            $count = $this->targetConnection->executeUpdate($stm);

            //delete notes for these cases
```

```
            $stm = 'DELETE FROM `notes` WHERE `case_id` in ( ' . $strCaseList
. ")";
            $this->targetConnection->executeUpdate($stm);

            //cascade delete cases from break out tables
            $stm = 'DELETE FROM `level-1` WHERE `case-id` in ( ' .
$strCaseList . ")";
            $count = $this->targetConnection->executeUpdate($stm);
            $this->logger->debug("Deleted $count cases");

            $this->targetConnection->commit();
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed deleting cases';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            $this->targetConnection->rollBack();
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

To :

```
    public function deleteQuestionnaires() {
        //delete all the questionnaires that match
        $caseList = array_keys($this->casesMap);
        $strCaseList = "'" . implode("','", $caseList) . "'";

        $this->targetConnection->beginTransaction();
        try {
            //delete existing cases
            $stm = 'DELETE FROM '.strtolower($this->labelDictionnaire).'_cases
WHERE id in ( ' . $strCaseList . ')';
            $count = $this->targetConnection->executeUpdate($stm);

            //delete notes for these cases
            $stm = 'DELETE FROM '.strtolower($this->labelDictionnaire).'_notes
WHERE case_id in ( ' . $strCaseList . ')';
            $this->targetConnection->executeUpdate($stm);

            //cascade delete cases from break out tables
            $stm = 'DELETE FROM "'.strtolower($this-
>labelDictionnaire).'_level-1" WHERE "case-id" in ( ' . $strCaseList . ')';
            $count = $this->targetConnection->executeUpdate($stm);
            $this->logger->debug("Deleted $count cases");
```

```
            $this->targetConnection->commit();
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed deleting cases';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            $this->targetConnection->rollBack();
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

- ✓ Transform « generateLevelInsertStatement » function :

From :

```
    private function generateLevelInsertStatement(&$nameTypeMap): string {
        $stm = "INSERT INTO `level-1` (";
        //TODO: fix for multiple levels
        $iLevel = 0;
        $level = $this->dict->getLevels()[$iLevel];

        for ($iItem = 0; $iItem < (is_countable($level->getIdItems()) ?
count($level->getIdItems()) : 0); $iItem++) {
            $this->getRecordItemNameType($level->getIdItems()[$iItem],
$nameTypeMap);
        }
        $keys = array_keys($nameTypeMap);
        $quotedItemNames = [];
        foreach ($keys as $key) {
            $quotedItemNames[] =
MySQLDictionarySchemaGenerator::quoteString($key);
        }
        $itemList = implode(",", $quotedItemNames);
        $itemList = "`case-id`," . $itemList;

        $stm .= $itemList . ") VALUES ";
        return $stm;
    }
```

To :

```
    private function generateLevelInsertStatement(&$nameTypeMap): string {
        $stm = 'INSERT INTO "'.strtolower($this->labelDictionnaire).'_level-1"
(';
        //TODO: fix for multiple levels
        $iLevel = 0;
```

```
        $level = $this->dict->getLevels()[$iLevel];
        for ($iItem = 0; $iItem < (is_countable($level->getIdItems()) ?
count($level->getIdItems()) : 0); $iItem++) {
            $this->getRecordItemNameType($level->getIdItems()[$iItem],
$nameTypeMap);
        }
        $keys = array_keys($nameTypeMap);
        $quotedItemNames = [];
        foreach ($keys as $key) {
            // $quotedItemNames[] =
MySQLDictionarySchemaGenerator::quoteString($key);
            $quotedItemNames[] = $key;
        }
        $itemList = implode(",", $quotedItemNames);
        $itemList = '"case-id",' . $itemList;

        $stm .= $itemList . ') VALUES ';
        return $stm;
    }
```

- ✓ Transform « generateRecordInsertStatement » function :

From :

```
    private function generateRecordInsertStatement(Record $record,
&$nameTypeMap): string {
        $recordName =
MySQLDictionarySchemaGenerator::quoteString(strtolower($record->getName()));
        $stm = "INSERT INTO $recordName (";

        $this->getRecordItemsNameType($record, $nameTypeMap);
        $keys = array_keys($nameTypeMap);
        $quotedItemNames = [];
        foreach ($keys as $key) {
            $quotedItemNames[] =
MySQLDictionarySchemaGenerator::quoteString($key);
        }

        $itemList = implode(",", $quotedItemNames);

        $parentLevelName = "level-" . (string) ($record->getLevel()-
>getLevelNumber() + 1);
        $parentId = $parentLevelName . "-id";

        if ($record->getMaxRecords() > 1) {
            $itemList = "`$parentId`, `occ`, " . $itemList;
        } else {
            $itemList = "`$parentId`," . $itemList;
```

```
        }
        $itemList = rtrim($itemList, ",");
        $stm .= $itemList . ") VALUES ";

        return $stm;
    }
```

To :

```
    private function generateRecordInsertStatement(Record $record,
&$nameTypeMap): string {
        $recordName = strtolower($this-
>labelDictionnaire)."_".strtolower($record->getName());
        $stm = "INSERT INTO $recordName (";

        $this->getRecordItemsNameType($record, $nameTypeMap);
        $keys = array_keys($nameTypeMap);
        $quotedItemNames = [];
        foreach ($keys as $key) {
            // $quotedItemNames[] =
MySQLDictionarySchemaGenerator::quoteString($key);
            $quotedItemNames[] = $key;
        }

        $itemList = implode(",", $quotedItemNames);

        $parentLevelName = "level-" . (string) ($record->getLevel()-
>getLevelNumber() + 1);
        $parentId = $parentLevelName . "-id";
        $parentId = '"'.$parentId.'"';

        if ($record->getMaxRecords() > 1) {
            $itemList = "$parentId, occ, " . $itemList;
        } else {
            $itemList = "$parentId," . $itemList;
        }
        $itemList = rtrim($itemList, ",");
        $stm .= $itemList . ") VALUES ";

        return $stm;
    }
```

- ✓ Transform « serializeCases » function :

From :

```
    public function serializeCases(): int {
```

```
        $caseList = array_keys($this->casesMap);
        $stm = "INSERT INTO `cases` (`id`, `key`, `label`,
`last_modified_revision`, `deleted`, `verified`, `partial_save_mode`,
`partial_save_field_name`, `partial_save_level_key`,
`partial_save_record_occurrence`, `partial_save_item_occurrence`, "
                . "                 `partial_save_subitem_occurrence`) VALUES
";
        $itemNames = ["guid", "caseids", "label", "revision", "deleted",
"verified", "partial_save_mode", "partial_save_field_name",
"partial_save_level_key", "partial_save_record_occurrence",
"partial_save_item_occurrence", "partial_save_subitem_occurrence"];
        $values = [];
        $singlePlaceholder = '(' . implode(', ', array_fill(0,
count($itemNames), '?')) . ')';
        // (?, ?), ... , (?, ?)
        $placeholders = implode(', ', array_fill(0, count($this->casesMap),
$singlePlaceholder));

        $this->logger->debug('Inserting into cases table: processing ' .
count($this->casesMap) . ' cases to insert');
        foreach ($caseList as $case) {
            $caseRow = $this->casesMap[$case];
            foreach ($itemNames as $itemName) {
                if (isset($caseRow[$itemName])) {
                    $values[] = $caseRow[$itemName];
                } else {
                    $values[] = null;
                }
            }
            //once the case is processed change the key in the map to point to
json decoded questionnaire for the
            //rest of the tables to be broken out
            $this->casesMap[$case] = json_decode($caseRow['questionnaire'],
true);
            if (json_last_error() != JSON_ERROR_NONE) {
                $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . '
TargetDB: ' . $this->targetConnection->getDatabase();
                $strMsg .= ' Dictionary: ' . $this->dict->getName() . "]
Failed writing cases to database for  jobID: " . $this->jobId;
                $strQuestionnaire = ' Case: ' . $case . ' Questionnaire: ' .
$caseRow['questionnaire'];
                $this->logger->error($strMsg . $strQuestionnaire . " Error
decoding json questionnaire. " . json_last_error_msg());
                throw new \Exception($strMsg);
            }
        }

        $stm .= $placeholders;
        try {
```

```
            $count = $this->targetConnection->executeUpdate($stm, $values);
            $this->logger->debug("inserted  $count rows into cases table");
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= "] Failed writing cases to database for  jobID: " .
$this->jobId;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
        return $count;
    }
```

To :

```
public function serializeCases(): int {
        $caseList = array_keys($this->casesMap);
        $stm = "INSERT INTO ".strtolower($this->labelDictionnaire)."_cases
(id, key, label, last_modified_revision, deleted, verified, partial_save_mode,
partial_save_field_name, partial_save_level_key,
partial_save_record_occurrence, partial_save_item_occurrence, "
                . "                 partial_save_subitem_occurrence) VALUES ";
        $itemNames = ["guid", "caseids", "label", "revision", "deleted",
"verified", "partial_save_mode", "partial_save_field_name",
"partial_save_level_key", "partial_save_record_occurrence",
"partial_save_item_occurrence", "partial_save_subitem_occurrence"];
        $values = [];
        $singlePlaceholder = '(' . implode(', ', array_fill(0,
count($itemNames), '?')) . ')';
        // (?, ?), ... , (?, ?)
        $placeholders = implode(', ', array_fill(0, count($this->casesMap),
$singlePlaceholder));

        $this->logger->debug('Inserting into cases table: processing ' .
count($this->casesMap) . ' cases to insert');
        foreach ($caseList as $case) {
            $caseRow = $this->casesMap[$case];
            foreach ($itemNames as $itemName) {
                if (isset($caseRow[$itemName])) {
                    $values[] = $caseRow[$itemName];
                } else {
                    $values[] = null;
                }
            }
            //once the case is processed change the key in the map to point to
json decoded questionnaire for the
            //rest of the tables to be broken out
```

```
            $this->casesMap[$case] = json_decode($caseRow['questionnaire'],
true);
            if (json_last_error() != JSON_ERROR_NONE) {
                $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . '
TargetDB: ' . $this->targetConnection->getDatabase();
                $strMsg .= ' Dictionary: ' . $this->dict->getName() . "]
Failed writing cases to database for  jobID: " . $this->jobId;
                $strQuestionnaire = ' Case: ' . $case . ' Questionnaire: ' .
$caseRow['questionnaire'];
                $this->logger->error($strMsg . $strQuestionnaire . " Error
decoding json questionnaire. " . json_last_error_msg());
                throw new \Exception($strMsg);
            }
        }

        $stm .= $placeholders;
        try {
            if(count($values) > 2100 ){
                $colonnes = floor(count($values)/count(array_fill(0,
count($this->casesMap), $singlePlaceholder)));
                if(1200 % $colonnes != 0){
                    $length_chunk = floor(1200 / $colonnes) * $colonnes;
                }else{
                    $length_chunk = 1200;
                }

                $chunkedValues = array_chunk($values, $length_chunk);
                $count = 0;
                foreach($chunkedValues as $chunkedValue){
                    $limited = (count($chunkedValue)/$colonnes);
                    $part_fill = array_slice(array_fill(0, count($this-
>casesMap), $singlePlaceholder), 0, $limited);
                    $stm2 = "INSERT INTO ".strtolower($this-
>labelDictionnaire)."_cases (id, key, label, last_modified_revision, deleted,
verified, partial_save_mode, partial_save_field_name, partial_save_level_key,
partial_save_record_occurrence, partial_save_item_occurrence, "
                    . "                 partial_save_subitem_occurrence)
VALUES ";
                            $itemNames = ["guid", "caseids", "label",
"revision", "deleted", "verified", "partial_save_mode",
"partial_save_field_name", "partial_save_level_key",
"partial_save_record_occurrence", "partial_save_item_occurrence",
                                "partial_save_subitem_occurrence"];

                    $placeholders2 = implode(', ', $part_fill);

                    $stm2 .= $placeholders2;
```

```
                    $count_with_chunk = $this->targetConnection-
>executeUpdate($stm2, $chunkedValue);
                    $count += $count_with_chunk;
                }
            }else{
                $count = $this->targetConnection->executeUpdate($stm,
$values);
            }
            $this->logger->debug("inserted  $count rows into cases table");
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= "] Failed writing cases to database for  jobID: " .
$this->jobId;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
        return $count;
    }
```

✓ Transform « serializeNotes » function :

From :

```
    public function serializeNotes(): int {
        $caseList = array_keys($this->casesMap);

        //select notes for the cases in case map from the source dictionary
notes table
        $sourceNotesTable = '`' . $this->dict->getName() . '_notes`';
        $stm = 'SELECT LCASE(CONCAT_WS("-", LEFT(HEX( `case_guid`), 8),
MID(HEX(`case_guid`), 9,4), MID(HEX( `case_guid`), 13,4), MID(HEX(
`case_guid`), 17,4), RIGHT(HEX( `case_guid`), 12))) as `case_id`, '
                . "`operator_id`, `field_name`, `level_key`,
`record_occurrence`, `item_occurrence`, `subitem_occurrence`, `content`,
`modified_time`   FROM " . $sourceNotesTable . ' WHERE case_guid IN ( ';

        $whereData = [];
        $n = 0;
        // prepare the where clause in list for all the case guids to delete
the notes for the correponding cases
        foreach ($caseList as $case) {
            $strWhere [] = 'UNHEX(REPLACE(' . ":case_guid$n" . ',"-",""))';
            $whereData ['case_guid' . $n] = $case;
            $n++;
        }

        if (!empty($strWhere)) {
```

```
            $stm .= implode(', ', $strWhere);
            $stm .= ' );';
        }

        try {
            $stmt = $this->sourcePdo->prepare($stm);
            $stmt->execute($whereData);

            $result = $stmt->fetchAll();

            if ((is_countable($result) ? count($result) : 0) == 0)
                return 0;

            //add the notes for these cases to  the notes table
            $stm = "INSERT INTO `notes` (`case_id`, `field_name`, `level_key`,
`record_occurrence`, `item_occurrence`, "
                    . "`subitem_occurrence`, `content`, `operator_id`,
`modified_time`) VALUES ";
            $itemNames = ["case_id", "field_name", "level_key",
"record_occurrence", "item_occurrence", "subitem_occurrence", "content",
"operator_id", "modified_time"];
            $values = [];
            $singlePlaceholder = '(' . implode(', ', array_fill(0,
count($itemNames), '?')) . ')';
            // (?, ?), ... , (?, ?)
            $placeholders = implode(', ', array_fill(0, is_countable($result)
? count($result) : 0, $singlePlaceholder));

            $this->logger->debug('Inserting into notes table');
            foreach ($result as $row) {
                foreach ($itemNames as $itemName) {
                    if (isset($row[$itemName])) {
                        if ($itemName === 'modified_time') {
                            $values[] = date('Y-m-d H:i:s',
strtotime($row['modified_time']));
                        } else {
                            $values[] = $row[$itemName];
                        }
                    } else {
                        $values[] = null;
                    }
                }
            }

            $stm .= $placeholders;
            $count = $this->targetConnection->executeUpdate($stm, $values);
            $this->logger->debug("inserted  $count notes");
        } catch (\Exception $e) {
```

```
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= "] Failed writing case notes to database for  jobID: "
. $this->jobId;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
        return $count;
    }
```

To :

```
    public function serializeNotes(): int {
        $caseList = array_keys($this->casesMap);

        //select notes for the cases in case map from the source dictionary
notes table
        $sourceNotesTable = '`' . $this->dict->getName() . '_notes`';
        $stm = 'SELECT LCASE(CONCAT_WS("-", LEFT(HEX( `case_guid`), 8),
MID(HEX(`case_guid`), 9,4), MID(HEX( `case_guid`), 13,4), MID(HEX(
`case_guid`), 17,4), RIGHT(HEX( `case_guid`), 12))) as `case_id`, '
                . "`operator_id`, `field_name`, `level_key`,
`record_occurrence`, `item_occurrence`, `subitem_occurrence`, `content`,
`modified_time`   FROM " . $sourceNotesTable . ' WHERE case_guid IN ( ';

        $whereData = [];
        $n = 0;
        // prepare the where clause in list for all the case guids to delete
the notes for the correponding cases
        foreach ($caseList as $case) {
            $strWhere [] = 'UNHEX(REPLACE(' . ":case_guid$n" . ',"-","")))';
            $whereData ['case_guid' . $n] = $case;
            $n++;
        }

        if (!empty($strWhere)) {
            $stm .= implode(', ', $strWhere);
            $stm .= ' );';
        }

        try {
            $stmt = $this->sourcePdo->prepare($stm);
            $stmt->execute($whereData);

            $result = $stmt->fetchAll();

            if ((is_countable($result) ? count($result) : 0) == 0)
                return 0;
```

```
            //add the notes for these cases to  the notes table
            $stm = "INSERT INTO ".strtolower($this->labelDictionnaire)."_notes
(case_id, field_name, level_key, record_occurrence, item_occurrence, "
                    . "subitem_occurrence, content, operator_id,
modified_time) VALUES ";
            $itemNames = ["case_id", "field_name", "level_key",
"record_occurrence", "item_occurrence", "subitem_occurrence", "content",
"operator_id", "modified_time"];
            $values = [];
            $singlePlaceholder = '(' . implode(', ', array_fill(0,
count($itemNames), '?')) . ')';
            // (?, ?), ... , (?, ?)
            $placeholders = implode(', ', array_fill(0, is_countable($result)
? count($result) : 0, $singlePlaceholder));

            $this->logger->debug('Inserting into notes table');
            foreach ($result as $row) {
                foreach ($itemNames as $itemName) {
                    if (isset($row[$itemName])) {
                        if ($itemName === 'modified_time') {
                            $values[] = date('Y-m-d H:i:s',
strtotime($row['modified_time']));
                        } else {
                            $values[] = $row[$itemName];
                        }
                    } else {
                        $values[] = null;
                    }
                }
            }

            $stm .= $placeholders;
            $count = $this->targetConnection->executeUpdate($stm, $values);
            $this->logger->debug("inserted  $count notes");
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= "] Failed writing case notes to database for  jobID: "
. $this->jobId;
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
        return $count;
    }
```

✓ Transform « getCaseIdsMap » function :

From :

```
    private function getCaseIdsMap() {
        try {
            // Select all the cases sent by the client that exist on the
server
            $stm = 'SELECT  `level-1-id` as id, `case-id` as guid FROM `level-
1` WHERE `case-id` in (';
            $strOrderBy = ' ORDER BY  id';

            $strCaseList = "'" . implode("','", array_keys($this->casesMap)) .
"'";
            $stm = $stm . $strCaseList . ")" . $strOrderBy;
            $result = $this->targetConnection->fetchAllAssociative($stm);

            $this->casesIdMap = [];
            foreach ($result as $row) {
                $this->casesIdMap[$row ['guid']] = $row['id'];
            }
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed getting cases to process for dictionary';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

To :

```
    private function getCaseIdsMap() {
        try {
            // Select all the cases sent by the client that exist on the
server
            $stm = 'SELECT  "level-1-id" as id, "case-id" as guid FROM
"'.strtolower($this->labelDictionnaire).'_level-1" WHERE "case-id" in (';
            $strOrderBy = ' ORDER BY  id';

            $strCaseList = "'" . implode("','", array_keys($this->casesMap)) .
"'";
            $stm = $stm . $strCaseList . ")" . $strOrderBy;
            $result = $this->targetConnection->fetchAllAssociative($stm);

            $this->casesIdMap = [];
            foreach ($result as $row) {
```

```
                $this->casesIdMap[$row ['guid']] = $row['id'];
            }
        } catch (\Exception $e) {
            $strMsg = '[SourceDB: ' . $this->sourcePdo->getDsn() . ' TargetDB:
' . $this->targetConnection->getDatabase();
            $strMsg .= ' Dictionary: ' . $this->dict->getName();
            $strMsg .= '] Failed getting cases to process for dictionary';
            $this->logger->error($strMsg, ["context" => (string) $e]);
            throw new \Exception($strMsg, 0, $e);
        }
    }
```

D. Add break-Out by dictionary

For adding « single dictionary break-out », it's necessary to create à new command class in « **src/AppBundle/command** » directory.

- ✓ Create a php file named « **CSWebProcessRunnerByDict.php** »
- ✓ File script :

```
<?php

namespace AppBundle\Command;

use Symfony\Component\Console\Command\Command;
use Symfony\Component\Console\Command\LockableTrait;
use Symfony\Component\Console\Exception\InvalidArgumentException;
use Symfony\Component\Console\Input\InputInterface;
use Symfony\Component\Console\Input\InputOption;
use Symfony\Component\Console\Output\OutputInterface;
use Symfony\Component\HttpKernel\KernelInterface;
use Symfony\Component\Process\Exception\RuntimeException;
use Symfony\Component\Process\PhpExecutableFinder;
use Symfony\Component\Process\Process;
use AppBundle\Service\PdoHelper;
use Psr\Log\LoggerInterface;
use AppBundle\CSPro\Dictionary\MySQLDictionarySchemaGenerator;
use AppBundle\CSPro\DictionaryHelper;
use AppBundle\CSPro\DictionarySchemaHelper;
use Doctrine\DBAL\Schema;
use Doctrine\DBAL\DriverManager;
use Doctrine\DBAL\Configuration;
use AppBundle\CSPro\DBConfigSettings;
use Symfony\Component\Console\Input\InputArgument;

/**
```

```
 * Description of CSWebProcessRunner - based on
https://stackoverflow.com/questions/54127418/backend-multi-threading-in-php-7-
symfony4
 *
 * @author savy
 */
class CSWebProcessRunnerByDict extends Command {

    protected static $defaultName = 'csweb:process-cases-by-dict';
    use LockableTrait;

    public const MAX_TIME_LIMIT = 3600; //seconds

    private $startTime;
    private $phpBinaryPath;
    private $dictionaryMap;
    private $maxCasesPerChunk;
    private $output;

    //TODO: eventually use  DBAL instead of PDO for all the service
operations.
    public function __construct(private PdoHelper $pdo, private
KernelInterface $kernel, private LoggerInterface $logger) {
        parent::__construct();
        $this->dictionaryMap = [];
    }

    protected function configure() {
        //configuration is set to running max three threads per dictionary
with each thread processing a a max of 500 cases
        $this->setDescription('CSWeb blob breakout processing into multiple
threads')
                ->addOption('threads', 't', InputOption::VALUE_REQUIRED,
'Number of threads to run at once per dictionary', 3)
                ->addOption('maxCasesPerChunk', 'c',
InputOption::VALUE_REQUIRED, 'Number of cases to process per chunk', 1000)
                ->addArgument('dictionnaires', InputArgument::IS_ARRAY,
'Tableau de dictionnaires cspro');
    }

    protected function execute(InputInterface $input, OutputInterface $output)
{
        $dictionnaires = $input->getArgument('dictionnaires');
        if (!$this->lock()) {
            $output->writeln('The command is already running in another
process.');
            $this->logger->info('The command is already running in another
process.');
            return 0;
```

```
        }
        $this->logger->info('Started process at ' . date("c"));
        $this->startTime = microtime(true);
        $this->output = $output;
        $this->maxCasesPerChunk = $input->getOption('maxCasesPerChunk');
        $threadsPerDictionary = $input->getOption('threads');
        $output->writeln('Running blob breakout process.');
        if (extension_loaded('pcntl')) {
            $stop = function () {
                $output = null;
                $this->logger->error('Abort process issued');
                $output->writeln('Abort process issued.');
                $output->writeln('Stopping blob breakout process.');
                throw new RuntimeException('Abort process issued');
            };
            pcntl_signal(SIGTERM, $stop);
            pcntl_signal(SIGINT, $stop);
            pcntl_async_signals(true);
        }
        //generate schema for each dictionary that is to be processed if it
does not exists
        $this->createDictionarySchemas($dictionnaires);
        $stopProcess = $this->hasProcessTimeExpired();
        do {
            try {
                //set stop process flag if process time expires and no threads
are running currently
                $stopProcess = count($this->dictionaryMap) == 0 || ($this-
>hasProcessTimeExpired() && $this->canExitProcess());
                foreach (array_keys($this->dictionaryMap) as $dictionaryName)
{

                    $dictionaryInfo = &$this->dictionaryMap[$dictionaryName];

                    if ($dictionaryInfo->processFlag == false &&
(is_countable($dictionaryInfo->processes) ? count($dictionaryInfo->processes)
: 0) == 0) {
                        //no jobs available and no running threads for this
dictionary. Remove dictionary from processing
                        unset($this->dictionaryMap[$dictionaryName]);
                        $this->logger->info("No jobs available to process for
dictionary: " . $dictionaryName);
                    }
                    //create new threads if duration is within process expiry
time.
                    while ((is_countable($dictionaryInfo->processes) ?
count($dictionaryInfo->processes) : 0) < $threadsPerDictionary && !$this-
>hasProcessTimeExpired() && $dictionaryInfo->processFlag) {
```

```
                        $output->writeln('Processing dictionary: ' .
$dictionaryName . '- Running threads ' . (is_countable($dictionaryInfo-
>processes) ? count($dictionaryInfo->processes) : 0));
                        $this->logger->debug('CSWeb Process Runner creating a
new blob breakout thread');
                        $output->writeln('creating a new blob breakout
thread');
                        $process = $this->createProcess($dictionaryName);
                        if ($process) {
                            $process->setTimeout(self::MAX_TIME_LIMIT);
                            $process->setIdleTimeout(self::MAX_TIME_LIMIT);
                            $process->start();
                            $dictionaryInfo->processes[] = $process;
                        }
                    }

                    //filters array and returns running processes for the
current dictionary
                    $dictionaryInfo->processes = array_filter($dictionaryInfo-
>processes, fn(Process $p) => $p->isRunning());
                }
                //For use to debug
                /* for ($j = 0; $j < count($dictionaryInfo->processes); $j++)
{
                  $dictionaryInfo->processes[$j]->wait(function ($type,
$buffer) {
                  echo 'OUT > ' . $type;
                  if (Process::ERR === $type) {
                  echo 'ERR > ' . $buffer;
                  } else {
                  echo 'OUT > ' . $buffer;
                  }
                  });
                  $this->output->writeln("removing process count is " .
count($dictionaryInfo->processes));
                  array_splice($dictionaryInfo->processes, $j, 1);
                  $this->output->writeln("after removal process count is " .
count($dictionaryInfo->processes));
                  } */
                sleep(1);
            } catch (RuntimeException) {
                try {
                    $this->output->writeln("killing process");
                    defined('SIGKILL') || define('SIGKILL', 9);
                    //kill the running threads
                    foreach (array_keys($this->dictionaryMap) as
$dictionaryName) {
                        $dictionaryInfo = &$this-
>dictionaryMap[$dictionaryName];
```

```
                            array_map(function (Process $p) {
                                $p->signal(SIGKILL);
                            }, $dictionaryInfo->processes);
                        }
                    } catch (\Throwable) {

                    }
                    break;
                }
            } while (!$stopProcess);
            $this->release();
            $this->logger->info('Stopping process at ' . date("c"));
            return 0;
        }

        private function createProcess($dictName) {
            // var_dump('HERE'); die;
            if (!isset($this->dictionaryMap[$dictName])) {
                $this->logger->error("Invalid dictionary Map. Dictionary
Information not set for dictionary " . $dictName);
                return null;
            }

            $dictionaryInfo = &$this->dictionaryMap[$dictName];
            $dictionarySchemaHelper = $dictionaryInfo->schemaHelper;
            $jobId = $dictionarySchemaHelper->processNextJob($this-
>maxCasesPerChunk);
            $this->output->writeln('Creating process for dictionary ' . $dictName
. ' jobID: ' . $jobId);
            $this->logger->debug('Creating process for dictionary ' . $dictName .
' jobID: ' . $jobId);
            if (!$this->phpBinaryPath) {
                $this->phpBinaryPath = (new PhpExecutableFinder())->find();
            }
            if ($jobId) {
                $cmd = [
                    $this->phpBinaryPath,
                    '-f',
                    realpath($this->kernel->getProjectDir() . '/bin/console'),
                    '--',
                    'csweb:blob-breakout-worker',
                    '-e',
                    $this->kernel->getEnvironment(),
                    '-d',
                    $dictName,
                    '-j',
                    $jobId,
                ];
```

```
            $this->output->writeln('Processing for dictionary ' . $dictName .
' jobID: ' . $jobId);
            $this->logger->debug('Processing for dictionary ' . $dictName . '
jobID: ' . $jobId);
            return new Process($cmd);
        }
        $this->output->writeln('No jobs available to run for dictionary: ' .
$dictName);
        //set process flag to false to stop creating threads for this
dictionary
        $dictionaryInfo->processFlag = false;
        return null;
    }

    private function createDictionarySchemas($tabDict) {

        $listDictName = "'DICT'";

        foreach($tabDict as $dict){
            $listDictName = $listDictName .",'".$dict."'";
        }

        //do exception handling
        $stm = 'SELECT id, dictionary_name as dictName FROM
`cspro_dictionaries` JOIN `cspro_dictionaries_schema`  ON dictionary_id =
cspro_dictionaries.id WHERE dictionary_name IN ('.$listDictName.')';

        $result = $this->pdo->fetchAll($stm);

        if (count($result) > 0) {
            $this->dictionaryMap = [];
            foreach ($result as $row) {
                $this->logger->info('Updating schema tables for Dictionary: '
. $row['dictName']);
                $dictionarySchemaHelper = new
DictionarySchemaHelper($row['dictName'], $this->pdo, $this->logger);
                $dictionarySchemaHelper->initialize(true);
                $dictionarySchemaHelper->resetInProcesssJobs();

                //set the dictionary information
                $dictionaryInfo = new \stdClass;
                $dictionaryInfo->schemaHelper = $dictionarySchemaHelper;
                $dictionaryInfo->processFlag = true;
                $dictionaryInfo->processes = [];

                $this->dictionaryMap[$row['dictName']] = $dictionaryInfo;
            }
        }
    }
```

```
    private function canExitProcess(): bool {
        $flag = true;
        foreach (array_keys($this->dictionaryMap) as $dictionaryName) {
            $dictionaryInfo = $this->dictionaryMap[$dictionaryName];
            $processes = $dictionaryInfo->processes;
            if (isset($processes) && (is_countable($processes) ?
count($processes) : 0) > 0) { //if threads are running return false;
                $flag = false;
                break;
            }
        }
        return $flag;
    }

    private function hasProcessTimeExpired() {
        $duration = round((microtime(true) - $this->startTime));
        return $duration > self::MAX_TIME_LIMIT;
    }

}
```

```
ALTER TABLE csweb_kairos.cspro_dictionaries_schema DROP KEY
schema_name;
```